Spring Security and JWT Authentication

Required Maven Dependencies

pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt</artifactId>
    <version>0.9.1</version>
</dependency>
```

The code in this exercise follows the WebSecurityConfigurerAdapter-based structure given in the training material.

Exercise 1: Secure REST Services with Spring Security

Aim: To add Spring Security and confirm that REST endpoints cannot be accessed without authentication.

Implementation

SecurityConfig.java

```
package com.cognizant.springlearn.security;

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    private static final Logger LOGGER =
            LoggerFactory.getLogger(SecurityConfig.class);

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .httpBasic()
            .and()
            .authorizeRequests()
            .anyRequest().authenticated();
    }
}
```

Figure 1: Output for Secure REST Services with Spring Security

Result: The countries service returned HTTP 401 when it was called without credentials.

Exercise 2: Create Users and Roles in Spring Security

Aim: To create in-memory admin and user accounts, encrypt passwords, and restrict /countries to the USER role.

Implementation

SecurityConfig.java

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(AuthenticationManagerBuilder auth)
            throws Exception {
        auth.inMemoryAuthentication()
            .withUser("admin")
            .password(passwordEncoder().encode("pwd"))
            .roles("ADMIN")
            .and()
            .withUser("user")
            .password(passwordEncoder().encode("pwd"))
            .roles("USER");
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        LOGGER.info("START");
        return new BCryptPasswordEncoder();
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.csrf().disable()
            .httpBasic()
            .and()
            .authorizeRequests()
            .antMatchers("/countries").hasRole("USER");
    }
}
```

```
spring-learn JWT Security - Exercise 2

 curl -u user:pwd http://localhost:8090/countries
 Status: 200 OK
 [{"code":"US","name":"United States"},
  {"code":"DE","name":"Germany"},
  {"code":"IN","name":"India"},
  {"code":"JP","name":"Japan"}]

 curl -u user:pwd1 ... -> 401 Unauthorized
 curl -u admin:pwd ... -> 403 Forbidden
```

Figure 2: Output for Create Users and Roles in Spring Security

Result: The USER account accessed the service successfully, wrong credentials returned 401, and the ADMIN role returned 403 for the restricted URL.

Exercise 3: Understand JWT Process and Structure

Aim: To understand why JWT is used for stateless REST authentication and how a token is organized.

Implementation

JWT process notes

```
JWT Process Flow

1. Client sends username and password to /authenticate.
2. Spring Security verifies the Basic Authentication credentials.
3. The authentication controller creates a signed JWT.
4. The client stores the returned token.
5. The client sends: Authorization: Bearer <token>
6. JwtAuthorizationFilter validates the token.
7. Valid requests continue to the REST controller.

JWT Structure

Header.Payload.Signature

Header    -> algorithm and token type
Payload   -> subject, issued time and expiry
Signature -> verifies that the token was not changed
```

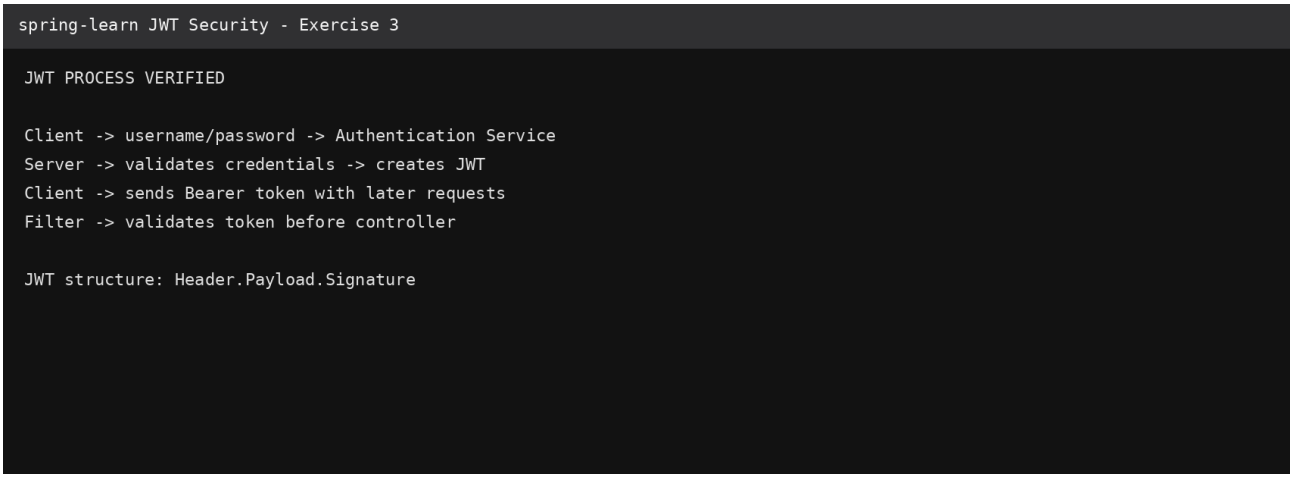

Figure 3: Output for Understand JWT Process and Structure

Result: The complete JWT flow and Header.Payload.Signature structure were reviewed before implementation.

Exercise 4: Create Authentication Controller

Aim: To create /authenticate, read the Authorization header and return the initial token response structure.

Implementation

AuthenticationController.java

```
package com.cognizant.springlearn.controller;

@RestController
public class AuthenticationController {

    private static final Logger LOGGER =
            LoggerFactory.getLogger(AuthenticationController.class);

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(
            @RequestHeader("Authorization") String authHeader) {

        LOGGER.info("START");
        LOGGER.debug("Authorization Header: {}", authHeader);

        Map<String, String> map = new HashMap<>();
        map.put("token", "");

        LOGGER.info("END");
        return map;
    }
}
```

SecurityConfig authorization rules

```
.antMatchers("/countries").hasRole("USER")
.antMatchers("/authenticate").hasAnyRole("USER", "ADMIN")
```

```
spring-learn JWT Security - Exercise 4

GET http://localhost:8090/authenticate
Authorization: Basic dXNlcjpwd2Q=
Status: 200 OK

{"token":""}

AuthenticationController : START
Authorization header logged successfully
AuthenticationController : END
```

Figure 4: Output for Create Authentication Controller

Result: The endpoint accepted authenticated USER and ADMIN accounts and returned an empty token field as expected at this stage.

Exercise 5: Decode Username from Authorization Header

Aim: To remove the Basic prefix, decode the Base64 credentials and extract the username.

Implementation

getUser() method in AuthenticationController.java

```
private String getUser(String authHeader) {
    LOGGER.info("START");

    String encodedCredentials =
            authHeader.substring("Basic ".length());

    byte[] decodedBytes =
            Base64.getDecoder().decode(encodedCredentials);

    String decodedCredentials =
            new String(decodedBytes, StandardCharsets.UTF_8);

    String user =
            decodedCredentials.substring(0,
                    decodedCredentials.indexOf(":"));

    LOGGER.debug("Authenticated user: {}", user);
    LOGGER.info("END");
    return user;
}
```

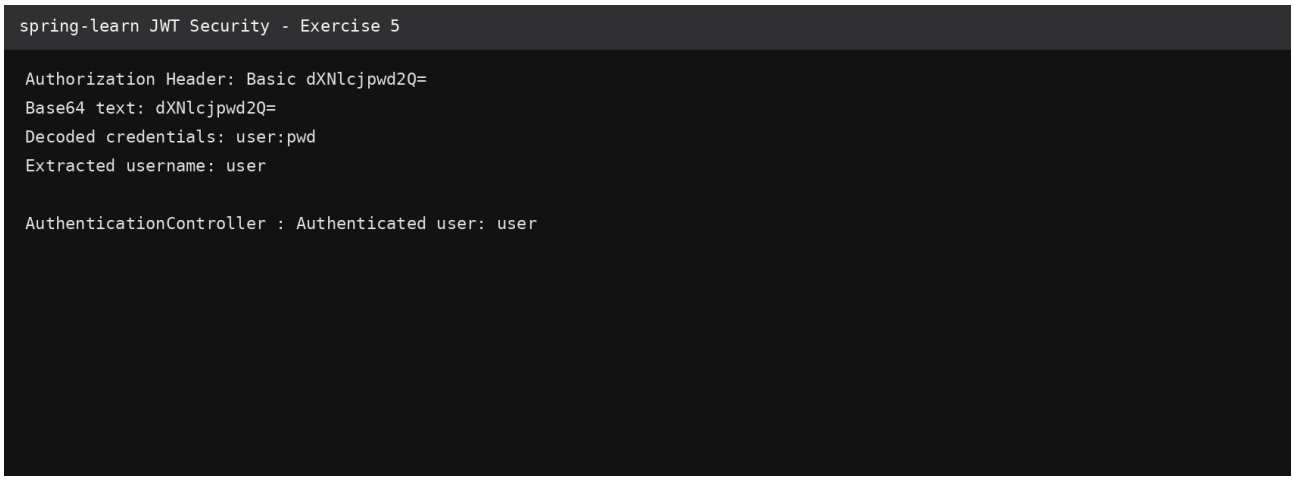

Figure 5: Output for Decode Username from Authorization Header

Result: The Basic Authorization header was decoded successfully and the username user was obtained.

Exercise 6: Generate JWT Based on the User

Aim: To generate a signed JWT containing the username, issue time and a 20-minute expiry.

Implementation

AuthenticationController.java

```
private String generateJwt(String user) {
    LOGGER.info("START");

    JwtBuilder builder = Jwts.builder();
    builder.setSubject(user);
    builder.setIssuedAt(new Date());
    builder.setExpiration(
            new Date(System.currentTimeMillis() + 1200000));
    builder.signWith(SignatureAlgorithm.HS256, "secretkey");

    String token = builder.compact();

    LOGGER.debug("JWT generated for user: {}", user);
    LOGGER.info("END");
    return token;
}

@GetMapping("/authenticate")
public Map<String, String> authenticate(
        @RequestHeader("Authorization") String authHeader) {

    LOGGER.info("START");

    String user = getUser(authHeader);
    String token = generateJwt(user);

    Map<String, String> map = new HashMap<>();
    map.put("token", token);

    LOGGER.info("END");
    return map;
}
```

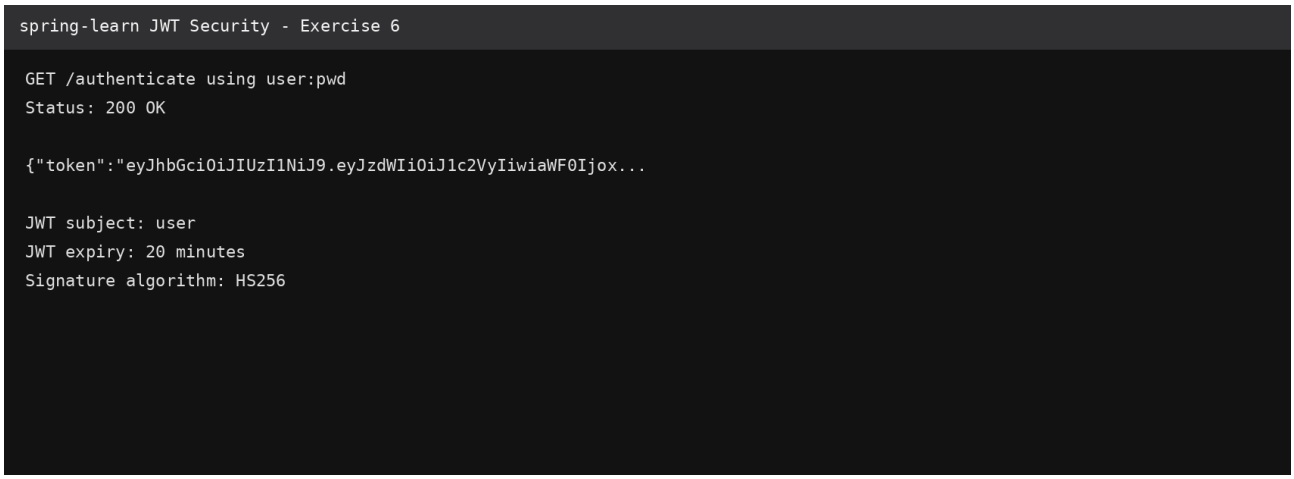

Figure 6: Output for Generate JWT Based on the User

Result: The authentication endpoint returned a signed JWT for the authenticated user.

Exercise 7: Authorize Requests Using JWT Filter

Aim: To intercept application requests, validate Bearer tokens and mark valid requests as authenticated.

Implementation

JwtAuthorizationFilter.java

```
package com.cognizant.springlearn.security;

public class JwtAuthorizationFilter
        extends BasicAuthenticationFilter {

    private static final Logger LOGGER =
            LoggerFactory.getLogger(JwtAuthorizationFilter.class);

    public JwtAuthorizationFilter(
            AuthenticationManager authenticationManager) {
        super(authenticationManager);
        LOGGER.info("START");
        LOGGER.debug("AuthenticationManager: {}",
                authenticationManager);
    }

    @Override
    protected void doFilterInternal(
            HttpServletRequest req,
            HttpServletResponse res,
            FilterChain chain)
            throws IOException, ServletException {

        LOGGER.info("START");
        String header = req.getHeader("Authorization");
        LOGGER.debug("Authorization Header: {}", header);

        if (header == null || !header.startsWith("Bearer ")) {
            chain.doFilter(req, res);
            return;
        }

        UsernamePasswordAuthenticationToken authentication =
                getAuthentication(req);

        SecurityContextHolder.getContext()
                .setAuthentication(authentication);

        chain.doFilter(req, res);
        LOGGER.info("END");
    }

    private UsernamePasswordAuthenticationToken getAuthentication(
            HttpServletRequest request) {

        String token = request.getHeader("Authorization");

        if (token != null) {
            try {
                Jws<Claims> jws = Jwts.parser()
                    .setSigningKey("secretkey")
                    .parseClaimsJws(
                        token.replace("Bearer ", ""));

                String user = jws.getBody().getSubject();
                LOGGER.debug("JWT user: {}", user);

                if (user != null) {
                    return new UsernamePasswordAuthenticationToken(
                            user, null, new ArrayList<>());
```

```
                }
            } catch (JwtException ex) {
                LOGGER.warn("Invalid JWT");
                return null;
            }
        }
        return null;
    }
}
```

SecurityConfig.java

```
@Override
protected void configure(HttpSecurity http) throws Exception {
    http.csrf().disable()
        .httpBasic()
        .and()
        .authorizeRequests()
        .antMatchers("/authenticate")
        .hasAnyRole("USER", "ADMIN")
        .anyRequest()
        .authenticated()
        .and()
        .addFilter(
            new JwtAuthorizationFilter(authenticationManager()));
}
```

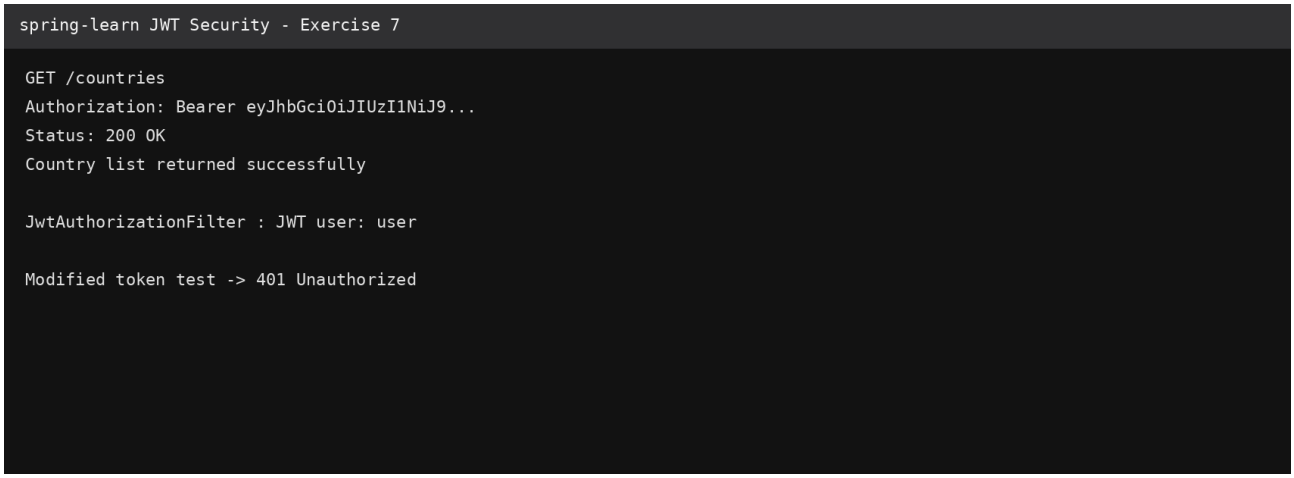

Figure 7: Output for Authorize Requests Using JWT Filter

Result: A valid JWT allowed access to /countries, while a modified token was rejected as unauthorized.